



CHECKPOINT 1 DESIGN DOCUMENT

Secure Web-Based File Encryption and Management System

System overview, architecture, authentication model, and threat model

Combined from the project README, authentication and session model, and threat model.

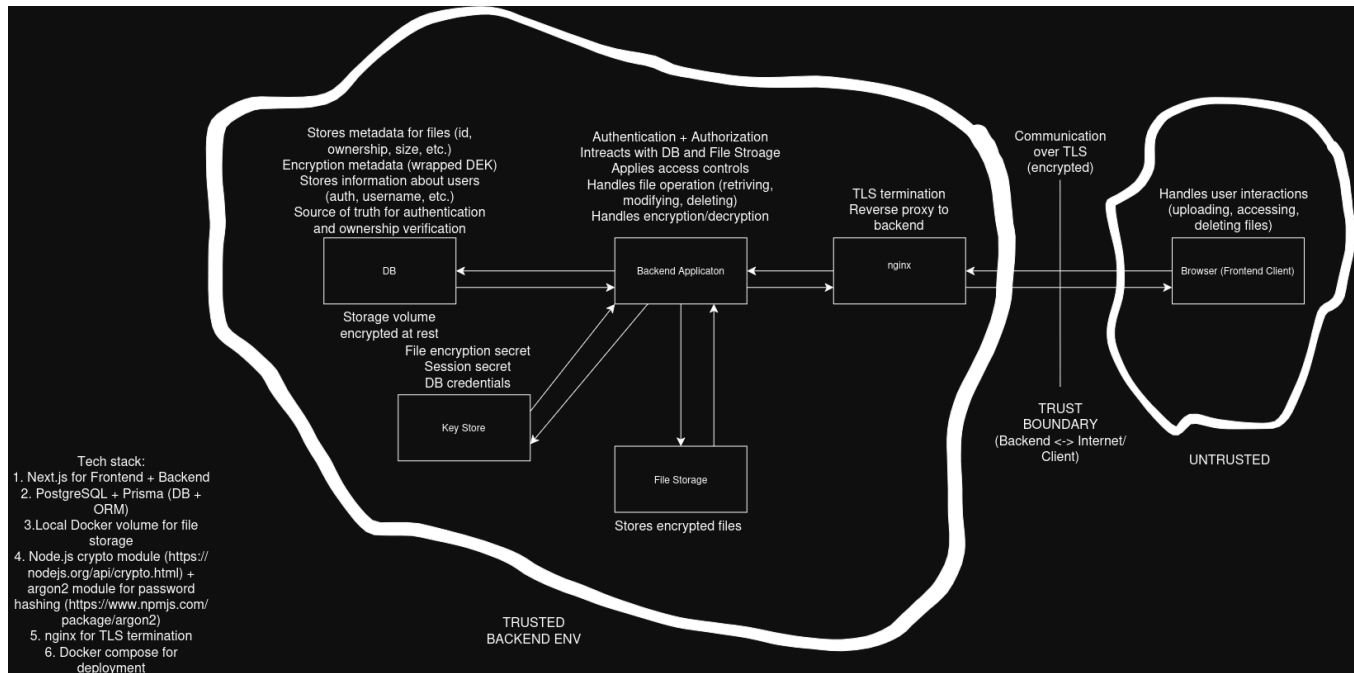
Contents

1. Project overview and architecture
2. Authentication and session model
3. Threat model

Project overview and architecture

This project is a web-based application designed to securely store and manage files for users. It provides encryption capabilities, allowing users to upload, access, and delete files securely. The system combines modern web technologies with strong cryptographic practices to ensure user privacy and data security.

System Architecture Diagram



Technology Stack

1. **Frontend & Backend: Next.js** Next.js is a modern framework that provides unified full-stack capabilities. It simplifies development by allowing both frontend and backend logic to be written in JavaScript or TypeScript. Its built-in route handlers eliminate the need for a separate backend application while maintaining a clean separation of concerns.
2. **Database: PostgreSQL + Prisma** PostgreSQL is a robust, reliable relational database that ensures data durability and consistency. Prisma serves as an ORM layer, providing type-safe database queries and automatic schema management. Together, they simplify database interactions while maintaining query performance and data integrity.
3. **File Storage: Docker Volume (Encrypted at Rest)** Using a Docker volume for file storage keeps data local and allows for persistent storage across container deployments. All files are encrypted before being written to disk, ensuring that even if the storage is compromised, the data remains protected.
4. **Encryption & Hashing: Node.js Crypto + argon2** The native Node.js crypto module handles file encryption and decryption. argon2 is used for password hashing, providing strong protection against offline password-cracking attacks compared with older password-hashing algorithms.

5. Web Server & TLS: **nginx** Nginx acts as a reverse proxy and terminates TLS connections. TLS protects data in transit between the browser and the server, while file encryption at rest is performed separately by the application.
6. Deployment: **Docker Compose** Docker Compose simplifies deployment by defining and orchestrating all services (application, database, nginx, volumes) in a single configuration file. This ensures consistency across different environments (development, testing, production).

Planned Features

- Registration and login using email and password
- Opaque, cookie-based sessions managed by Better Auth
- Uploading files over TLS and encrypting them before they are written to storage
- Listing and searching only the authenticated user's files
- Downloading and decrypting files after a server-side ownership check
- Deleting files owned by the authenticated user
- Per-user storage quotas and upload-size limits
- Structured security logging without passwords, session tokens, encryption keys, or file contents

Planned Routes

User-facing routes

Route	Access	Purpose
/	Public	Landing page; redirects authenticated users to <code>/files</code> .
/login	Public	Email-and-password login page.
/register	Public	Account-registration page.
/files	Authenticated	File-management page for listing, searching, uploading, downloading, and deleting the current user's files.

Application API

Method	Route	Access	Purpose
GET	/api/health	Public	Reports whether the application is running. It must not disclose configuration, dependency, or secret information.
GET	/api/files	Authenticated	Lists the current user's file metadata. An optional <code>search</code> query parameter filters by filename.

Method	Route	Access	Purpose
POST	/api/files	Authenticated	Uploads a file using multipart/form-data . The backend validates the request, encrypts the file, and stores its metadata.
GET	/api/files/{id}	Authenticated owner	Checks ownership, decrypts the requested file, and returns it as a download.
DELETE	/api/files/{id}	Authenticated owner	Checks ownership and deletes the file and its associated metadata.

Every file endpoint validates the session on the server. Routes containing `{id}` use an opaque, server-generated file identifier and verify that the file's `ownerId` matches the authenticated user's ID.

Authentication API

Better Auth will be mounted using the recommended Next.js catch-all route at `/api/auth/[...all]` . The main operations used by this application are:

Method	Effective route	Purpose
POST	/api/auth/sign-up/email	Registers a user with an email address and password.
POST	/api/auth/sign-in/email	Authenticates a user and creates a server-side session.
POST	/api/auth/sign-out	Revokes the current session and clears its cookie.
GET	/api/auth/get-session	Returns the current authenticated session, if one exists.

The catch-all handler may expose additional Better Auth framework endpoints, but they are not part of the application's public feature set unless explicitly enabled and documented.

Running the Application Locally

The application will be runnable locally using Docker Compose. Detailed setup instructions, environment-variable requirements, TLS configuration, and startup commands will be added as the application is developed.

Security & Threat Model

For detailed information about the security architecture, threat analysis, and mitigation strategies implemented in this system, see [THREAT_MODEL.md](#) and [AUTHENTICATION_AND_SESSION_MODEL.md](#).

Authorship note: I wrote the content of this document. AI assistance was used to improve its wording and structure.

Authentication and session model

This documents present the authentication and session model for secure file storage application.

Library

In this project authentication is implemented using Better Auth. It's a framework agnostic library for TypeScript. It has a built-in support for secure email and password authentication, socia sign-on, built-in rate limiter, automatic DB management and configurable session security settings (<https://better-auth.com/docs/introduction>). Next.js creators recommend using such frameworks for implementing auth solutions:

"While you can implement a custom auth solution, for increased security and simplicity, we recommend using an authentication library. These offer built-in solutions for authentication, session management, and authorization, as well as additional features such as social logins, multi-factor authentication, and role-based access control. You can find a list in the Auth Libraries section."

Source: <https://nextjs.org/docs/app/guides/authentication>

Authentication Model & Session Management

Email/password credentials model is used for this project. Password are hashed using Argon2id algorithm and never storred/loggen in plaintext. After successful registration or login, Better Auth handles creation of session record in database and sends the client an opaque, signed session identifier in a cookie. This cookies doesn't contain any sensitive information. Information about the session is stored in the correpsonding database record. This record contains the id of the user whom the session belongs, expiration time, creation timestamp and some client metadata such as the IP address and user agent.

Sesssion Cookie

Session cookie is configured with the following attributes: - `HttpOnly` : ensures that session cookie can't be accessed, read or modified by client-side scripts - `Secure` : ensure that the cookies can only sent over HTTPS - `Same-Site=Strict` : prevents browser from sending the cookies with cross-site requests. - `Path=/` : makes the session cookie available to application routes

Session identifiers are accepted only from cookies. We never accept them through URL parameters or request bodies. Authentcation and CSRF(Origin checks, Fetch Metadata Protection for First-Login CSRF) checks provided by Better Auth remain enabled.

More about CSRF protection and authentication/session management provided by BetterAuth: <https://better-auth.com/docs/reference/security>

Session Lifetime and Revocation

Each session have a lifetime of 24 hours. Automatic sliding session refresh is disabled, so a session cannot remain active indefinitely. Logging out deletes the server-side session and clears the browser cookie. Sessions are also revoked after password reset. Cookie-based session caching is disabled so that revocation takes effect on the next request.

Session Fixation Prevention

The application never upgrades an existing anonymous or client-supplied session identifier into an authenticated session. Successful login and registration create a new server-generated session with a new unpredictable token, which replaces any previous session cookie.

After security-sensitive authentication changes, such as password reset, existing sessions are revoked and a new login is required. Session identifiers are accepted only through the protected session cookie and are never read from URLs. These measures prevent an attacker from selecting a session identifier and causing a victim to authenticate into that session.

Every protected page, Server Action, and Route Handler validates the session on the server. File access additionally checks that the authenticated user's ID matches the file's `ownerId` ; possession of a valid session does not grant access to another user's files.

Authorship note: I wrote the content of this document. AI assistance was used to improve its wording and structure.

Threat model

System description and scope

The system is a web-based encrypted file manager. Users authenticate through a web browser and can upload, download, list, and delete their files. Files are transferred from the browser over TLS, encrypted by the backend, and then written to file storage. The database stores user account information, file and ownership metadata, and encryption metadata.

The browser is considered an untrusted environment. Nginx, the backend application, the database, and file storage run in the server-controlled environment. They are treated as separate components because a compromise of each component has different consequences. For example, compromise of the storage volume should expose only encrypted file contents, whereas compromise of the backend may expose plaintext files and encryption keys.

Security objectives

- **Confidentiality:** files, credentials, keys, sessions, and sensitive metadata are disclosed only to authorized users and components.
- **Integrity:** files, metadata, sessions, and security-relevant records cannot be modified without authorization or without detection.
- **Availability:** authorized users can access the application and their files when required.
- **Auditability:** security-relevant actions can be attributed and investigated using complete, accurate, and protected logs.

Assets

The main assets of this application are:

1. **Uploaded files** - the contents of users' uploaded files
2. **User credentials and identity data** - email addresses, passwords during authentication, password hashes, account identities, and recovery material such as password-reset codes or links
3. **Authenticated sessions** - cookies, session IDs, server-side session records, and user identities bound to sessions
4. **Encryption keys and key material** - file-encryption keys, DEK/KEK material, and token-signing secrets
5. **Application secrets** - database credentials, TLS private keys, and other deployment secrets
6. **Database contents** - file metadata, ownership metadata, encryption metadata, and user information
7. **Service availability and resource integrity** - file-storage capacity and backend CPU, memory, and network resources
8. **Application logs** - log records, error details, and audit trails

Possible entry points and attack surface

1. **Frontend/browser UI** (login, registration, file upload, and file access pages) - DOM manipulation, XSS, bypassing disabled controls, and changing hidden fields
2. **API endpoints** (login, registration, and file upload, download, deletion, listing, and search endpoints) - parameter fuzzing, IDOR, SQL injection, and authorization bypass
3. **HTTP request bodies and headers** - malformed input, unexpected fields, fuzzed values, and spoofed proxy headers
4. **URL paths and query parameters** - path traversal, IDOR, and unexpectedly long or malformed values
5. **Cookies and session state** - session hijacking, replay, tampering, and authentication or authorization bypass
6. **Logging and error handling** - information disclosure, secrets in logs, log injection, and exposed stack traces
7. **Reverse proxy/Nginx** - misconfiguration, request-smuggling or request-forgery issues, and incorrect validation of forwarded headers

Threat table

No.	Threat	Description / attack example	Affected assets	Security objectives	OWASP category	Planned mitigation
1	Unauthorized file access or modification	A user changes a file identifier in an API request and attempts to read, replace, or delete a file owned by another user.	Uploaded files; database contents	Confidentiality, Integrity, Availability	A01:2025 Broken Access Control	Authenticate every request and perform a server-side ownership check for every file and metadata operation. Deny access unless the authenticated user's ID matches the file's <code>ownerId</code> .
2	Stored XSS through a filename	An attacker crafts a filename containing active HTML or JavaScript, such as <code><script>alert(1)</script>.pdf</code> , which could execute when the filename is displayed.	Authenticated sessions; uploaded files; database contents	Confidentiality, Integrity	A05:2025 Injection	Rely on React's contextual output escaping, validate filename length and permitted characters, and deploy a restrictive Content Security Policy. Do not render filenames as raw HTML.
3	Denial of service through oversized uploads	A user uploads an excessively large file, exhausting memory, temporary storage, bandwidth, or persistent storage.	Service availability and resource integrity	Availability	A06:2025 Insecure Design	Enforce request-size limits at Nginx and in the backend, verify the actual number of bytes received, stream uploads instead of buffering them in memory, and enforce per-user storage quotas. Client-side limits are for usability only.
4	File-search endpoint accepting excessively long input	A user submits an arbitrarily long search value, causing excessive memory use, expensive database queries, log amplification, or degraded service.	Service availability and resource integrity; application logs	Availability	A06:2025 Insecure Design	Enforce a reasonable maximum search length on the backend, reject malformed values early, use bounded database queries, and apply rate limiting where necessary.
5	SQL injection	Attacker-controlled search terms, filenames, or file IDs reach a dynamically constructed SQL query and alter its meaning.	Database contents; uploaded files; application logs	Confidentiality, Integrity, Availability, Auditability	A05:2025 Injection	Use Prisma's parameterized query APIs and prepared statements. Never concatenate user-controlled values into SQL, and prohibit unsafe raw-query APIs unless the query has been separately reviewed.
6	Password database compromise	An attacker obtains password hashes from the database and attempts offline password cracking.	User credentials and identity data	Confidentiality, Integrity	A04:2025 Cryptographic Failures	Hash passwords with Argon2id using unique salts and appropriately tuned memory, iteration, and parallelism parameters. Periodically review the parameters and increase their cost as deployment resources allow.

No.	Threat	Description / attack example	Affected assets	Security objectives	OWASP category	Planned mitigation
7	Cryptographic-key exposure	Encryption keys or signing secrets are committed to the repository, included in a container image, exposed to unnecessary processes, or written to logs.	Encryption keys and key material; application secrets; uploaded files	Confidentiality, Integrity	A04:2025 Cryptographic Failures	Inject secrets through the deployment environment or a secret manager. Never commit them to version control, include them in images, log them, or expose them to unnecessary processes. If a local <code>.env</code> file is used for development, exclude it from version control and restrict access to it.
8	Information leakage through errors	The backend returns stack traces, database errors, filesystem paths, or other internal implementation details to the client.	Application secrets; database contents; application logs	Confidentiality	A10:2025 Mishandling of Exceptional Conditions	Return generic client-facing errors and handle failures consistently. Record useful diagnostic details only in server-side logs, redact sensitive values, and associate client errors with non-sensitive correlation IDs.
9	File-storage compromise	An attacker obtains direct access to the raw file-storage volume and reads or tampers with stored ciphertext.	Uploaded files; encryption metadata	Confidentiality	A04:2025 Cryptographic Failures	Encrypt files before writing them to storage using authenticated encryption such as AES-GCM. Use a unique nonce for every encryption operation and verify the authentication tag before releasing plaintext. Storage may contain non-secret metadata such as nonces, authentication tags, key versions, or wrapped data-encryption keys, but it must not contain plaintext keys or material sufficient to decrypt files without the separately protected KEK.
10	Sensitive information in logs	Passwords, session tokens, encryption keys, file contents, or other sensitive values are accidentally written to logs. Log injection or tampering may also make the audit trail unreliable.	User credentials and identity data; authenticated sessions; application secrets; encryption keys and key material; application logs	Confidentiality, Auditability	A09:2025 Security Logging and Alerting Failures	Use structured logging, encode untrusted values, and apply sensitive-data filters before records are written. Restrict access to logs and protect them from unauthorized modification or deletion.
11	Session theft	An attacker obtains a session cookie and uses it to impersonate an authenticated user.	Authenticated sessions; uploaded files	Confidentiality, Integrity	A07:2025 Authentication Failures	Transmit session cookies only over TLS and set <code>HttpOnly</code> , <code>Secure</code> , and <code>SameSite=Strict</code> . Use unpredictable opaque session identifiers, a limited session lifetime, and server-side revocation on logout and password reset.
12	CSRF file deletion or modification	An attacker causes an authenticated user's browser to send an unwanted state-changing request to the application.	Uploaded files; database contents	Integrity, Availability	A01:2025 Broken Access Control	Keep Better Auth's Origin and Fetch Metadata checks enabled, use <code>SameSite=Strict</code> session cookies, and apply CSRF protection to every state-changing request. Never perform state changes through safe HTTP methods such as <code>GET</code> .
13	Path traversal	An attacker supplies a filename or path such as <code>../../../../other-user-file</code> to read, overwrite, or delete data outside the intended storage location.	Uploaded files; application secrets; service availability and resource integrity	Confidentiality, Integrity, Availability	A01:2025 Broken Access Control	Never use a user-supplied filename as a filesystem path. Store files under server-generated identifiers, keep original filenames only as metadata, resolve paths against a fixed storage root, and reject any path that escapes that root.
14	Brute-force login	An attacker repeatedly guesses a user's password or performs credential-stuffing attacks using credentials leaked from another service.	User credentials and identity data; authenticated sessions; service availability and resource integrity	Confidentiality, Integrity, Availability	A07:2025 Authentication Failures	Apply rate limits per account and source address, add progressive delays or temporary lockouts, return generic authentication errors that do not reveal whether an account exists, and log and alert on sustained failures.
15	Incomplete-upload cleanup and excessive concurrent uploads	Aborted uploads leave temporary or partially encrypted files behind, or a user starts many uploads concurrently to bypass quotas and exhaust file descriptors, memory, bandwidth, or storage.	Service availability and resource integrity; uploaded files; database contents	Integrity, Availability	A10:2025 Mishandling of Exceptional Conditions	Limit concurrent uploads and aggregate in-flight bytes per user, reserve quota before accepting an upload, write to a temporary server-generated path, and publish the file atomically only after encryption and validation succeed. Remove temporary data on failure and periodically clean up stale uploads.

No.	Threat	Description / attack example	Affected assets	Security objectives	OWASP category	Planned mitigation
16	Race conditions in file operations	Concurrent upload, download, and deletion requests exploit a gap between an authorization or existence check and the corresponding database or storage operation, causing quota bypass, inconsistent metadata, or operations on the wrong file.	Uploaded files; database contents; service availability and resource integrity	Confidentiality, Integrity, Availability	A10:2025 Mishandling of Exceptional Conditions	Use database transactions, unique constraints, atomic filesystem operations, and immutable server-generated file IDs. Repeat authorization checks within the operation's transaction, make retryable operations idempotent, and serialize conflicting operations on the same file where necessary.

Authorship note: I wrote the content of this document. AI assistance was used to improve its wording and structure.